

AI IMAGE CAPTIONING SYSTEM USING DEEP LEARNING

Rishika Reddy Thumma

1. Project Objective

The objective of this project is to develop and evaluate deep learning models that automatically generate descriptive captions for images, and to identify the architecture that produces the most reliable and accurate results.

Accurate image captioning has practical value beyond the modeling exercise itself: it improves accessibility for visually impaired users, supports better content organization, and enables more effective search and retrieval across digital libraries and online platforms.

2. Dataset and Preprocessing

Data Source

Initial experiments used the MS COCO dataset, with captions cleaned, tokenized, and encoded, and images scaled and normalized. A custom Coco Dataset class handled data loading, paired with an RNN decoder built on LSTM and a CNN encoder built on ResNet-50; a collate function managed caption padding, and PyTorch data loaders handled batching and shuffling.

The MS COCO training set totals roughly 19.5 GB, which exceeded the memory available in the coding environments used for this project. To keep experimentation practical, subsequent work moved to the Flickr30k dataset, which reduced memory and storage requirements while still providing a sufficiently large and diverse set of images and captions for training.

Feature Extraction and Caption Cleaning

For the Flickr30k experiments, image features were extracted using a pre-trained InceptionV3 network. Captions were cleaned and tokenized, then converted into word-index sequences, padded, and paired with their corresponding image features to form training examples.

For the final round of experiments, pre-extracted image features were loaded from a serialized pickle file for efficient reuse. Captions were parsed from a raw text file into an image-to-caption mapping, associating each image ID with its list of reference captions. This mapping was essential for both training and evaluation, since it allowed all captions for a given image to be retrieved together.

Each caption was then cleaned and standardized through the following steps:

- Lowercasing all text for uniformity.
- Removing digits, punctuation, and special characters, keeping only letters and spaces.
- Collapsing repeated spaces and trimming leading/trailing whitespace.
- Adding start and end markers (startseq / endseq) so the model could learn caption boundaries during training.

Preparing Data for Training

All cleaned training captions were collected into a single list and tokenized using Keras's Tokenizer class, converting words into numerical tokens. Vocabulary size was derived from the number of unique tokens (plus one, for padding), and the maximum sequence length was set to the length of the longest caption so that all input sequences could be padded to a consistent size.

The dataset was split into training and test sets, and a data generator produced batches on demand: for each image ID, it retrieved the associated captions, encoded them into sequences, and yielded image-feature/text-sequence input pairs together with the corresponding next-word labels. This kept memory use manageable while supporting scalable training.

3. Baseline Architecture

All models in this project share a common encoder-decoder design, differing only in how the text sequence is processed. The shared components are:

Image Feature Processing

A pre-trained CNN supplies fixed-length image feature vectors. These pass through a dropout layer to reduce overfitting, then a dense layer with ReLU activation projects them into a shared 256-dimensional representation used for caption generation.

Text Sequence Processing

Tokenized, padded caption sequences are mapped to 256-dimensional vectors through an embedding layer, followed by dropout for regularization. A recurrent layer then processes the sequence to capture word-to-word dependencies; this is the component that varies across the model experiments below (LSTM, GRU, or a bidirectional variant).

Feature Combination and Output

The image and text representations are merged with an addition layer, refined through a dense layer, and passed to a final softmax output layer that predicts the probability of each vocabulary word being the next word in the caption. Training minimizes categorical cross-entropy loss using the Adam optimizer.

4. Model Experiments

LSTM

The first model pairs the baseline text encoder with a standard LSTM layer to capture sequential dependencies between words. The model was trained for 10 epochs with a batch size of 64, minimizing categorical cross-entropy loss with the Adam optimizer.

Model: "model"

Layer (type)	Output Shape	Param #	Connected to
text (InputLayer)	[(None, 74)]	0	[]
image (InputLayer)	[(None, 4096)]	0	[]
embedding (Embedding)	(None, 74, 256)	5056000	['text[0][0]']
dropout (Dropout)	(None, 4096)	0	['image[0][0]']
dropout_1 (Dropout)	(None, 74, 256)	0	['embedding[0][0]']
dense (Dense)	(None, 256)	1048832	['dropout[0][0]']
lstm (LSTM)	(None, 256)	525312	['dropout_1[0][0]']
add (Add)	(None, 256)	0	['dense[0][0]', 'lstm[0][0]']
dense_1 (Dense)	(None, 256)	65792	['add[0][0]']
dense_2 (Dense)	(None, 19750)	5075750	['dense_1[0][0]']
...			
Total params: 11771686 (44.91 MB)			
Trainable params: 11771686 (44.91 MB)			
Non-trainable params: 0 (0.00 Byte)			

LSTM model architecture summary.

```

446/446 [=====] - 199s 431ms/step - loss: 4.9723
446/446 [=====] - 188s 421ms/step - loss: 3.9497
446/446 [=====] - 196s 438ms/step - loss: 3.6471
446/446 [=====] - 193s 432ms/step - loss: 3.4603
446/446 [=====] - 189s 423ms/step - loss: 3.3256
446/446 [=====] - 192s 431ms/step - loss: 3.2222
446/446 [=====] - 192s 429ms/step - loss: 3.1426
446/446 [=====] - 192s 430ms/step - loss: 3.0789
446/446 [=====] - 191s 429ms/step - loss: 3.0268
446/446 [=====] - 189s 424ms/step - loss: 2.9810
/usr/local/lib/python3.10/dist-packages/keras/src/engine/training.py:3103:
saving_api.save_model(

```

Training loss over 10 epochs — LSTM model.

Training loss decreased steadily from 4.9723 to 2.9810 over the 10 epochs, indicating effective learning.

```

100%|██████████| 3179/3179 [50:15<00:00, 1.05it/s]
BLEU-1: 0.442823
BLEU-2: 0.238008

```

BLEU-1 and BLEU-2 scores — LSTM model.

GRU

The second model replaces the LSTM layer with a Gated Recurrent Unit (GRU), which processes sequential caption data with a lighter parameter footprint than LSTM while following the same encoder-decoder structure. Training again ran for 10 epochs at a batch size of 64.

Model: "model_2"

Layer (type)	Output Shape	Param #	Connected to
text (InputLayer)	[(None, 74)]	0	[]
image (InputLayer)	[(None, 4096)]	0	[]
embedding_2 (Embedding)	(None, 74, 256)	5056000	['text[0][0]']
dropout_4 (Dropout)	(None, 4096)	0	['image[0][0]']
dropout_5 (Dropout)	(None, 74, 256)	0	['embedding_2[0][0]']
dense_6 (Dense)	(None, 256)	1048832	['dropout_4[0][0]']
gru (GRU)	(None, 256)	394752	['dropout_5[0][0]']
add_2 (Add)	(None, 256)	0	['dense_6[0][0]', 'gru[0][0]']
dense_7 (Dense)	(None, 256)	65792	['add_2[0][0]']
dense_8 (Dense)	(None, 19750)	5075750	['dense_7[0][0]']
...			
Total params: 11641126 (44.41 MB)			
Trainable params: 11641126 (44.41 MB)			
Non-trainable params: 0 (0.00 Byte)			

GRU-based model architecture summary.

446/446	[=====]	- 194s 425ms/step - loss: 4.7969
446/446	[=====]	- 189s 424ms/step - loss: 3.8480
446/446	[=====]	- 188s 420ms/step - loss: 3.5728
446/446	[=====]	- 184s 412ms/step - loss: 3.3997
446/446	[=====]	- 192s 430ms/step - loss: 3.2717
446/446	[=====]	- 191s 429ms/step - loss: 3.1713
446/446	[=====]	- 188s 423ms/step - loss: 3.0932
446/446	[=====]	- 189s 423ms/step - loss: 3.0324
446/446	[=====]	- 190s 426ms/step - loss: 2.9824
446/446	[=====]	- 187s 419ms/step - loss: 2.9385

Training loss over 10 epochs — GRU model.

Training loss decreased from 4.7969 to 2.9385 across the 10 epochs, showing steady improvement comparable to the LSTM model.

```
100%|██████████| 3179/3179 [48:24<00:00, 1.09it/s]
BLEU-1: 0.428156
BLEU-2: 0.231673
```

BLEU-1 and BLEU-2 scores — GRU model.

Transformer

An earlier phase of experimentation on the Flickr30k dataset compared three architectures — Transformer, GRU, and LSTM — using InceptionV3-extracted image features, with hyperparameter tuning applied to each configuration. Of these, the Transformer architecture produced the strongest results, reaching a BLEU-4 score of

0.323 and outperforming the GRU and LSTM configurations evaluated at that stage. This result motivated continued exploration of recurrent and bidirectional architectures in the final round of experiments below.

Convolutional-Bidirectional Model

The final model combines the CNN image encoder with a bidirectional LSTM text encoder. Image features (4096-dimensional) pass through a dropout layer (50%) and a dense layer (512 units, ReLU). Caption sequences pass through an embedding layer (256 dimensions, with masking for padding), dropout (50%), and two bidirectional LSTM layers of 256 units each, allowing the model to capture sequential context from both directions. The combined image and text representations are refined through a dense layer (512 units, ReLU) with dropout (50%) before the softmax output layer.

Model: "model_3"

Layer (type)	Output Shape	Param #	Connected to
text (InputLayer)	[(None, 74)]	0	[]
embedding_4 (Embedding)	(None, 74, 256)	5056000	['text[0][0]']
image (InputLayer)	[(None, 4096)]	0	[]
dropout_12 (Dropout)	(None, 74, 256)	0	['embedding_4[0][0]']
dropout_10 (Dropout)	(None, 4096)	0	['image[0][0]']
bidirectional_2 (Bidirectional)	(None, 74, 512)	1050624	['dropout_12[0][0]']
dense_11 (Dense)	(None, 512)	2097664	['dropout_10[0][0]']
dropout_13 (Dropout)	(None, 74, 512)	0	['bidirectional_2[0][0]']
dropout_11 (Dropout)	(None, 512)	0	['dense_11[0][0]']
bidirectional_3 (Bidirectional)	(None, 512)	1574912	['dropout_13[0][0]']
...			
Total params: 20698918 (78.96 MB)			
Trainable params: 20698918 (78.96 MB)			
Non-trainable params: 0 (0.00 Byte)			

Convolutional-Bidirectional model architecture summary.

The model was trained using the fit method, with input data, epochs, steps per epoch, validation data, validation steps, and verbosity all set as training parameters.

446/446	[=====]	- 288s 609ms/step	- loss: 5.1389
446/446	[=====]	- 259s 580ms/step	- loss: 4.3075
446/446	[=====]	- 259s 580ms/step	- loss: 4.0538
446/446	[=====]	- 259s 580ms/step	- loss: 3.8979
446/446	[=====]	- 258s 579ms/step	- loss: 3.7860
446/446	[=====]	- 258s 578ms/step	- loss: 3.7015
446/446	[=====]	- 259s 581ms/step	- loss: 3.6320
446/446	[=====]	- 259s 581ms/step	- loss: 3.5742
446/446	[=====]	- 258s 577ms/step	- loss: 3.5242
446/446	[=====]	- 258s 579ms/step	- loss: 3.4807

Training loss over 10 epochs — Convolutional-Bidirectional model.

Training loss decreased from 5.1389 to 3.4807 across the 10 epochs.

```
100%|██████████| 3179/3179 [1:11:29<00:00, 1.35s/it]
BLEU-1: 0.449615
BLEU-2: 0.243766
```

BLEU-1 and BLEU-2 scores — Convolutional-Bidirectional model.

5. Evaluation

BLEU Scores

Model quality was assessed using BLEU scores computed with NLTK's `corpus_bleu` function, comparing generated captions against the human-written reference captions in the test set. BLEU-1 measures single-word (unigram) overlap, while BLEU-2 measures bigram overlap and better reflects local word-order accuracy.

Captions were generated with a decoding routine that starts from the `startseq` token and predicts one word at a time — conditioned on the image features and the words generated so far — until it produces `endseq` or reaches the maximum caption length; the start and end tokens are then stripped from the final output.

Training Loss

Model	Initial Loss	Final Loss (Epoch 10)
LSTM	4.9723	2.9810
GRU	4.7969	2.9385
Convolutional-Bidirectional	5.1389	3.4807

All three final-phase models showed consistent, steady loss reduction over 10 epochs of training, with no signs of divergence.

6. Results

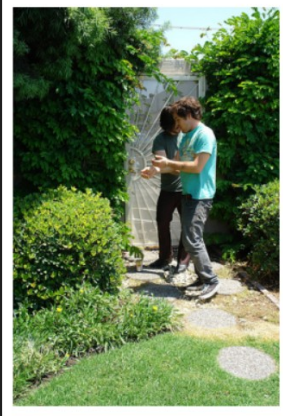
Model	BLEU-1	BLEU-2
LSTM	0.4428	0.2380
GRU	0.4282	0.2317
Convolutional-Bidirectional	0.4496	0.2438

Among the three architectures evaluated in this final phase, the Convolutional-Bidirectional model achieved the highest BLEU-1 and BLEU-2 scores, making it the most effective architecture for this task. The standard LSTM model performed competitively, trailing only slightly behind. The GRU model produced the lowest scores of the three, indicating it was the least effective option for this particular dataset and setup.

7. Sample Predictions

The trained Convolutional-Bidirectional model was used to generate captions for sample test images, shown below alongside their human-written reference captions.

Actual captions
 startseq two young guys with shaggy hair look at their hands while hanging out in the yard endseq
 startseq two young white males are outside near many bushes endseq
 startseq two men in green shirts are standing in yard endseq
 startseq man in blue shirt standing in garden endseq
 startseq two friends enjoy time spent together endseq
 Predicted caption
 man in blue shirt and jeans is walking down the street



Example 1 — reference captions vs. model-predicted caption.

Actual captions
 startseq two children girl and boy are practicing their writing endseq
 startseq two children sit on small seesaw in the sand endseq
 startseq two children sitting on teeter totter endseq
 startseq kids playing on seesaw endseq
 startseq two kids sit on seesaw endseq
 Predicted caption
 two men are playing in the sand



Example 2 — reference captions vs. model-predicted caption.



Example 3 — reference captions vs. model-predicted caption.

As a final check, the model was also tested on a new image outside the Flickr30k test set (a standalone JPG file), producing an accurate, contextually appropriate caption:



Prediction on an external image not seen during training.

8. Conclusion

This project developed and compared multiple deep learning architectures for automatic image captioning. The Convolutional-Bidirectional LSTM model delivered the strongest overall performance, achieving the highest BLEU-1 and BLEU-2 scores among the models evaluated in the final phase, with the standard LSTM model close behind and the GRU model trailing both. An earlier Transformer-based experiment on the same dataset also showed strong potential, reaching a BLEU-4 score of 0.323.

Overall, the results demonstrate that combining convolutional image features with bidirectional sequence modeling produces the most contextually accurate captions among the approaches tested. Future work could explore additional architectures, larger training datasets, and more advanced training strategies — such as attention mechanisms or further Transformer-based tuning — to continue improving caption accuracy and robustness.

9. References

Denotation Graph — Flickr30k dataset: <http://shannon.cs.illinois.edu/DenotationGraph/data/index.html>

IEEE Xplore: <https://ieeexplore.ieee.org/document/10088345>

TensorFlow Keras GRU layer: https://www.tensorflow.org/api_docs/python/tf/keras/layers/GRU

TensorFlow Keras LSTM layer: https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM

TensorFlow Keras Bidirectional layer: https://www.tensorflow.org/api_docs/python/tf/keras/layers/Bidirectional

TensorFlow Keras ImageDataGenerator: https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image/ImageDataGenerator

scikit-learn train_test_split:

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

NLTK BLEU score module: https://www.nltk.org/_modules/nltk/translate/bleu_score.html

scikit-learn model persistence: https://scikit-learn.org/stable/model_persistence.html